SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO3: *Analyze shape representation methods and techniques such as contours, regions, deformable models, and multi-resolution analysis. (BL4)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

EXPERIMENT

Code of Conduct:

I __Vallapu Mohan Kumar (192524225) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Docume ntat ion	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Annexure A

EXPERIMENT

Experiment 1: Contour-Based Representation & Fourier Descriptors

Scenario: Represent 2D shapes using contours and Fourier descriptors.

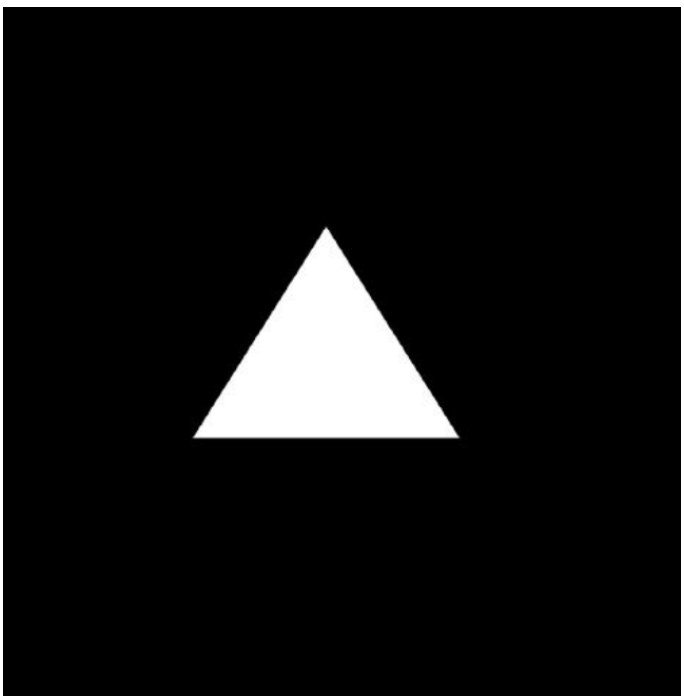
Input Data:

A. Binary images of 2D hand-drawn letters (A–Z) or simple symbols (triangle, square, star)

B. Image size: 256 × 256 pixels

C. File format: PNG or BMP

INPUT IMAGE:

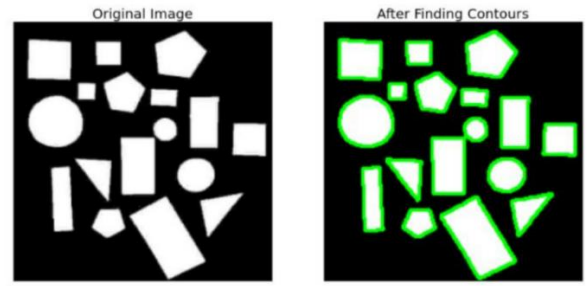


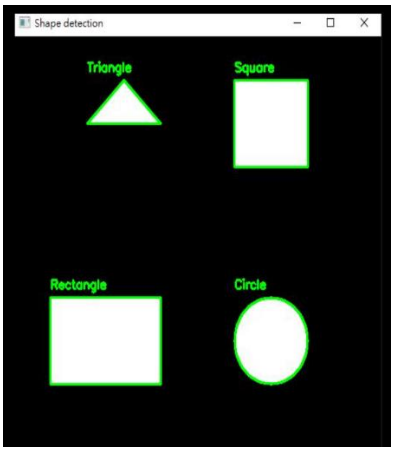
Python Code:

```
import cv2

import numpy as np
```

```
import matplotlib.pyplot as plt

image = cv2.imread("shape.png", 0)

_, binary = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)

if len(contours) == 0:

    print("No contours found")

    exit()

contour = max(contours, key=cv2.contourArea)

output = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)

cv2.drawContours(output, [contour], -1, (0, 255, 0), 2)

pts = contour[:, 0, :]


complex_pts = pts[:, 0] + 1j * pts[:, 1]


fd = np.fft.fft(complex_pts)

plt.figure(figsize=(12,5))

plt.subplot(121)

plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))

plt.title("Contour Detection")

plt.axis("off")


plt.subplot(122)

plt.plot(np.abs(fd))

plt.title("Fourier Descriptors")

plt.xlabel("Frequency")

plt.ylabel("Magnitude")


plt.show()
```

OUTPUT:





Experiment 2: Region-Based Representation & Medial Axis
Scenario: Analyze object shapes in binary images using medial axis. **Input Data:**

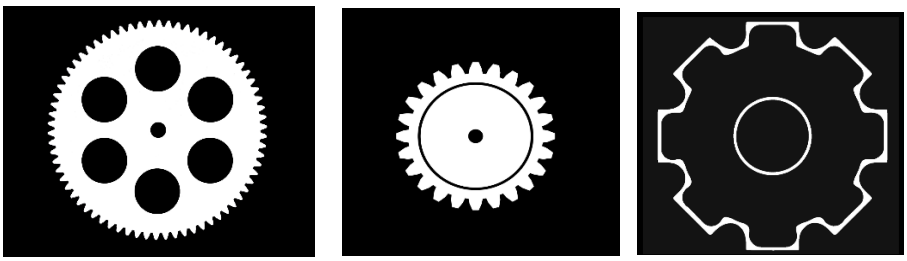
A. Binary images of industrial components (e.g., gears, machine parts)

B Image size: 512 × 512 pixels

C. Defects like small holes, protrusions, or missing parts included

D. File format: PNG

INPUT:



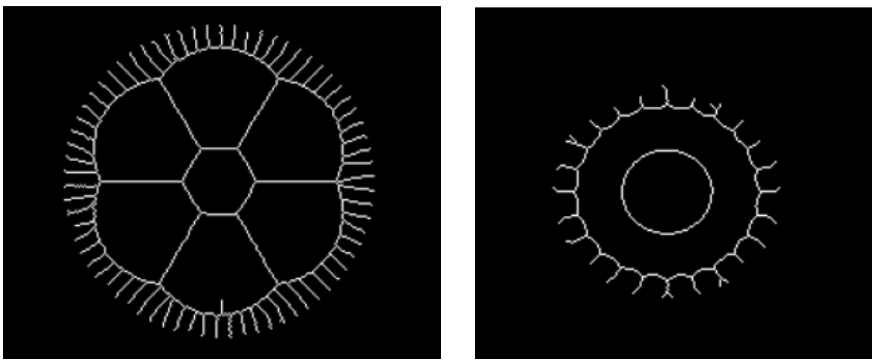
Python Code:

```
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.morphology import medial_axis

img = np.zeros((512,512), dtype=np.uint8)

cv2.circle(img, (256,256), 150, 255, -1)
cv2.circle(img, (256,256), 60, 0, -1)

for i in range(12):
    angle = np.deg2rad(i*30)
    x1 = int(256 + 150*np.cos(angle))
    y1 = int(256 + 150*np.sin(angle))
    x2 = int(256 + 190*np.cos(angle))
    y2 = int(256 + 190*np.sin(angle))
    cv2.line(img, (x1,y1), (x2,y2), 255, 15)

cv2.imwrite("gear.png", img)

image = cv2.imread("gear.png", 0)

_, binary = cv2.threshold(image, 127, 255,
cv2.THRESH_BINARY)
```

```
skeleton = medial_axis(binary > 0)


plt.figure(figsize=(10,5))


plt.subplot(1,2,1)
plt.imshow(binary, cmap="gray")
plt.title("Input Image")
plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(skeleton, cmap="gray")
plt.title("Medial Axis")
plt.axis("off")
plt.show()
```

Output:



Experiment 3: Deformable Curves and Snakes (Active Contours) Scenario: Segment organs or objects with irregular boundaries. **Input Data:**

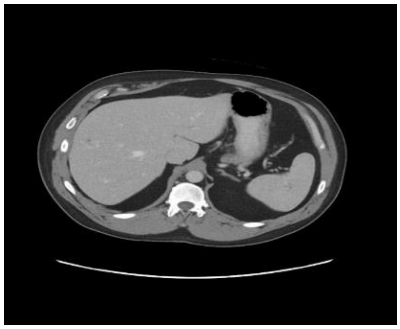
A. Grayscale medical images (e.g., liver or heart slices)

B. Image size: 512 × 512 pixels

C. Noise: mild Gaussian noise

D. File format: DICOM or PNG

Input:

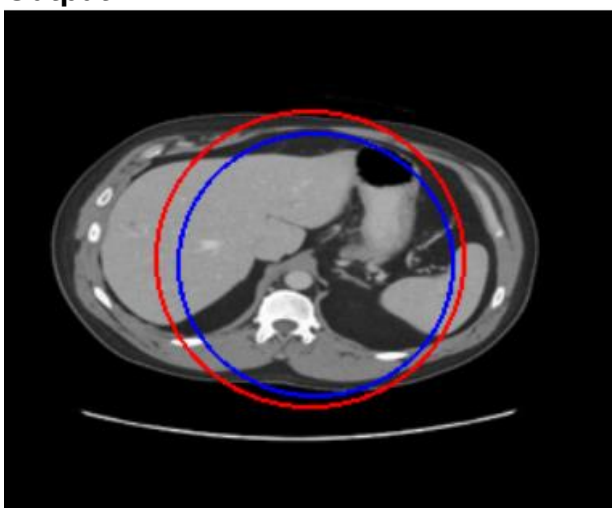


Python Code:

```
import numpy as np

import matplotlib.pyplot as plt

from skimage.io import imread

from skimage.filters import gaussian

from skimage.segmentation import active_contour


image = imread("/mnt/data/96271869-28c8-41d9-a075-c543918ca84f.png",
as_gray=True)


image = gaussian(image, sigma=3)


s = np.linspace(0, 2*np.pi, 400)


x = 140 + 90*np.cos(s)


y = 120 + 70*np.sin(s)
```

```
init = np.array([y, x]).T

snake = active_contour(image, init, alpha=0.015, beta=10, gamma=0.001)

plt.figure(figsize=(8,8))

plt.imshow(image, cmap="gray")

plt.plot(init[:,1], init[:,0], '--r', linewidth=2)

plt.plot(snake[:,1], snake[:,0], '-b', linewidth=2)

plt.title("Active Contour (Snake)")

plt.axis("off")

plt.show()
```

Output:



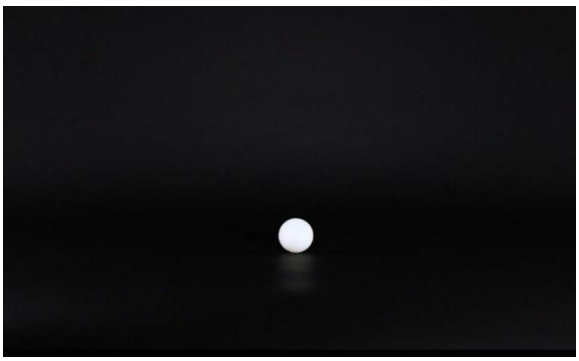
Experiment 4: Level Set Methods
Scenario: Track moving objects with evolving boundaries.
Input Data:

A. Video sequence (30 frames) of a moving object (e.g., a bouncing ball or moving hand)

B. Resolution: 640 × 480 pixels

C. Format: MP4 or sequence of PNG images

Input:

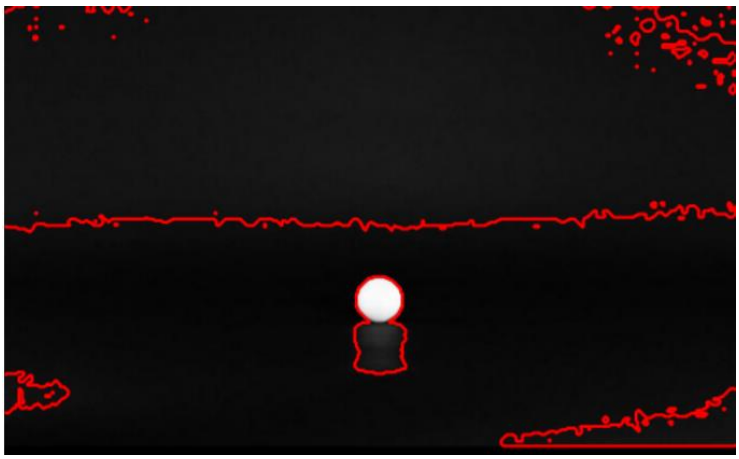


Python code:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from skimage.segmentation import morphological_chan_vese

from skimage.color import rgb2gray


image = cv2.imread("/mnt/data/a1020fbb-e034-465a-8ff3-cc1db42aad3b.png")


gray = rgb2gray(image)


levelset = morphological_chan_vese(gray, num_iter=100)


plt.figure(figsize=(10,5))
```

```
plt.subplot(1,2,1)
plt.imshow(gray, cmap="gray")
plt.title("Input Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(gray, cmap="gray")
plt.contour(levelset, [0.5], colors="red")
plt.title("Level Set Segmentation")
plt.axis("off")

plt.show()
```

Output:



Experiment 5: Multi-Resolution & Wavelet Descriptors

Scenario: Shape recognition at multiple scales.

Input Data:

A. Grayscale images of objects at different scales (e.g., leaves, mechanical parts, coins)

B. Image sizes: 128 × 128, 256 × 256, 512 × 512 pixels

C. Format: PNG

Input:

Python Code:

```
import cv2

import pywt

import matplotlib.pyplot as plt

image = cv2.imread("/mnt/data/66449ff3-eabc-45d4-a2c0-0a3da1df7d8f.png", 0)

image = cv2.resize(image, (512, 512))

coeffs2 = pywt.dwt2(image, 'haar')

LL, (LH, HL, HH) = coeffs2

plt.figure(figsize=(10,8))

plt.subplot(2,2,1)

plt.imshow(LL, cmap='gray')

plt.title("Approximation (LL)")

plt.axis("off")


plt.subplot(2,2,2)
```

```
plt.imshow(LH, cmap='gray')
plt.title("Horizontal (LH)")
plt.axis("off")

plt.subplot(2,2,3)
plt.imshow(HL, cmap='gray')
plt.title("Vertical (HL)")
plt.axis("off")

plt.subplot(2,2,4)
plt.imshow(HH, cmap='gray')
plt.title("Diagonal (HH)")
plt.axis("off")

plt.show()
```

Output: